

Guardrails and Prompt Caching

Understanding Safety and Efficiency in AI Agents

A model that generates high-quality outputs but not safe, too slow and expensive to run will not be useful

Guardrails make AI safe, caching makes AI fast

About Me



Sagar Chitransh

Senior Consultant | AI Engineer III

Deloitte

IIT Jodhpur (Rajasthan) Alumnus



AI Engineering Architecture

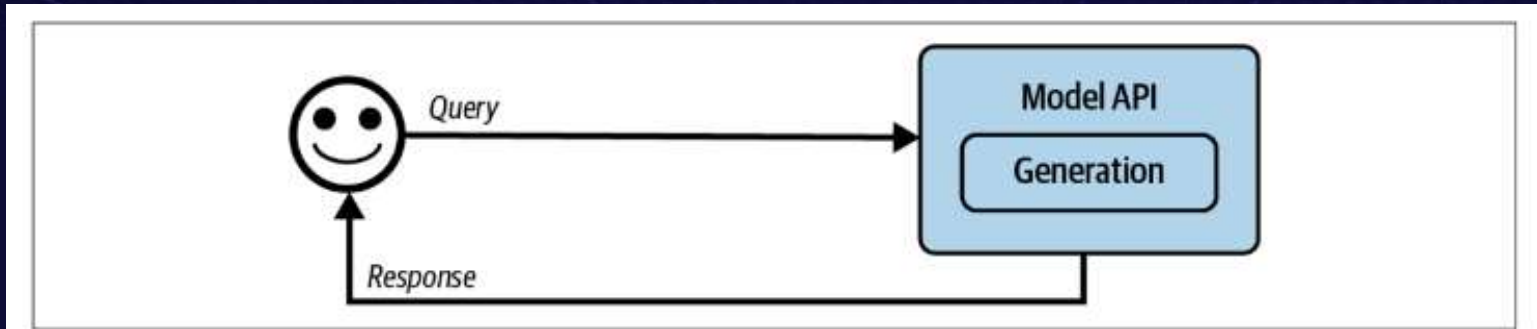


Figure 10-1. The simplest architecture for running an AI application.

MVP

1. Enhance context input into a model by giving the model access to external data sources and tools for information gathering.
2. Put in guardrails to protect your system and your users.
3. Add model router and gateway to support complex pipelines and add more security.
4. Optimize for latency and costs with caching.
5. Add complex logic and write actions to maximize your system's capabilities.

Source : AI Engineering
O'REILLY

Real Time Examples

- 1 An example is when a Samsung employee put Samsung's proprietary information into ChatGPT, accidentally **leaking the company's secrets.**

AI Platforms & Assistants > OpenAI > ChatGPT

Samsung workers made a major error by using ChatGPT

News By Lewis Maddison published 4 April 2023

Samsung meeting notes and new source code are now in the wild after being leaked in ChatGPT



(Image credit: Valeriy Zankovych / Shutterstock.com)

SAFETY CHALLENGES

Confidential



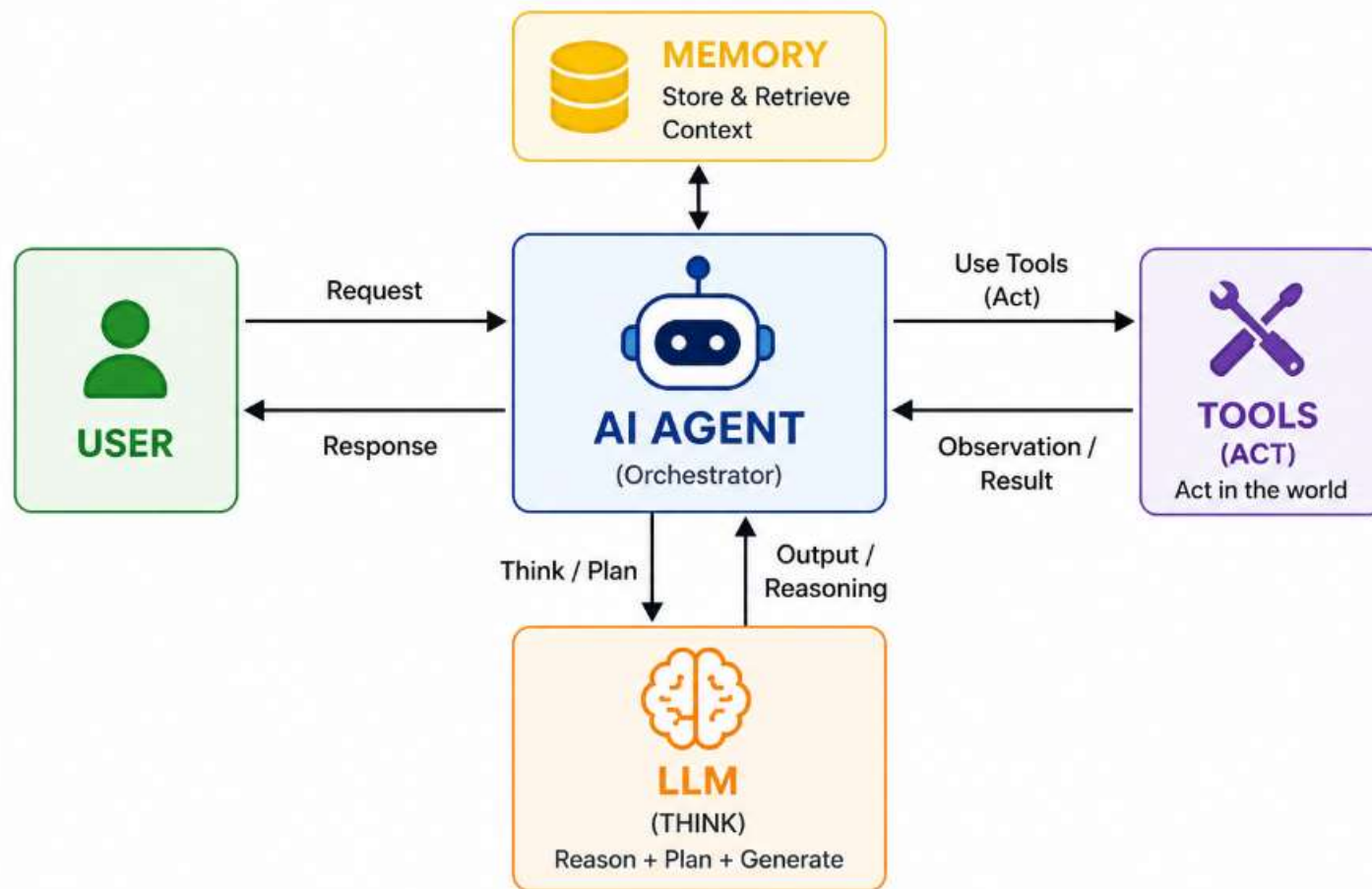
COST/BUDGET CHALLENGES

Source : AI Engineering

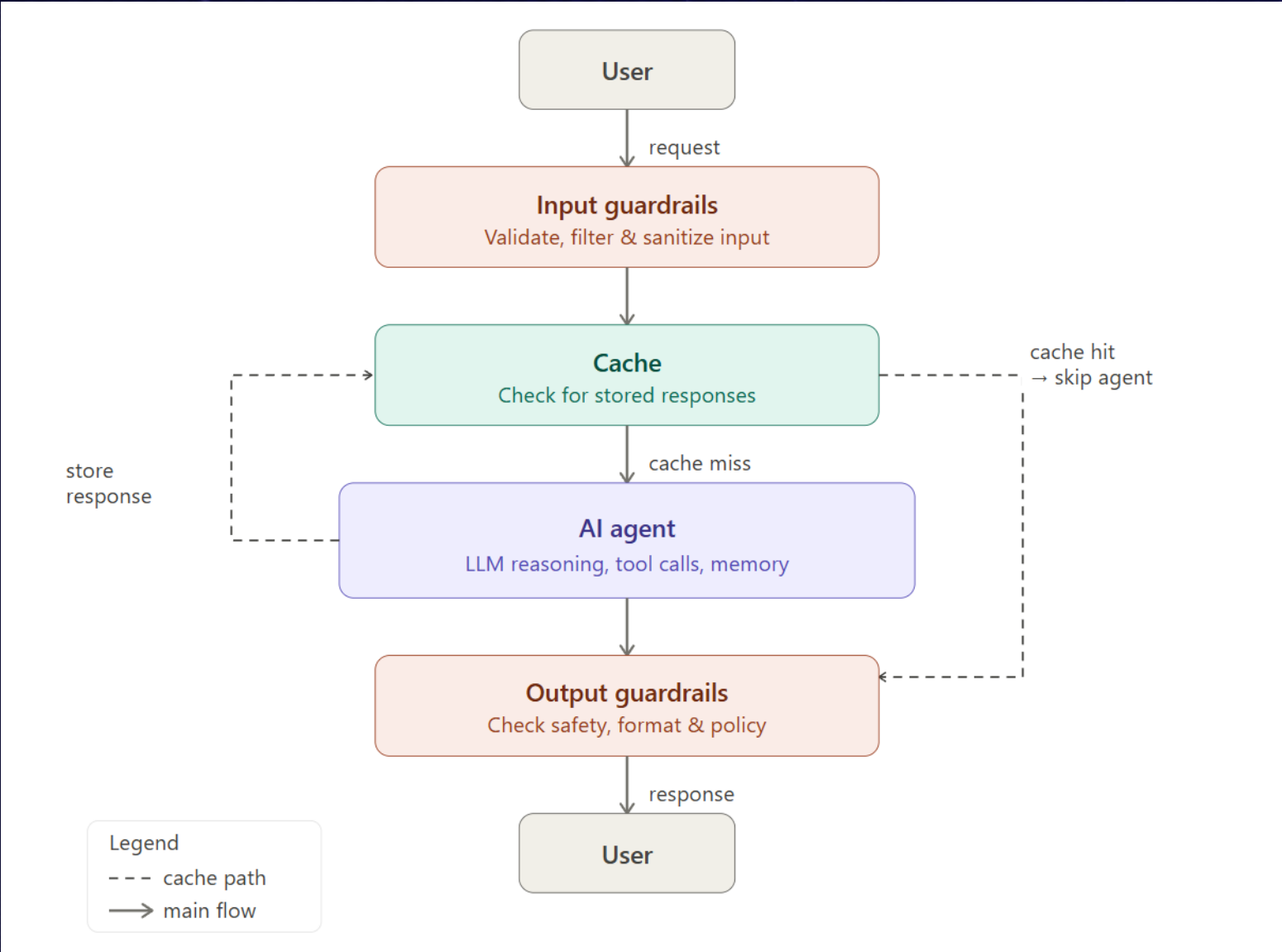
O'REILLY

Codebasics INC

AI AGENT ARCHITECTURE



Block Diagram showing guardrails and caching

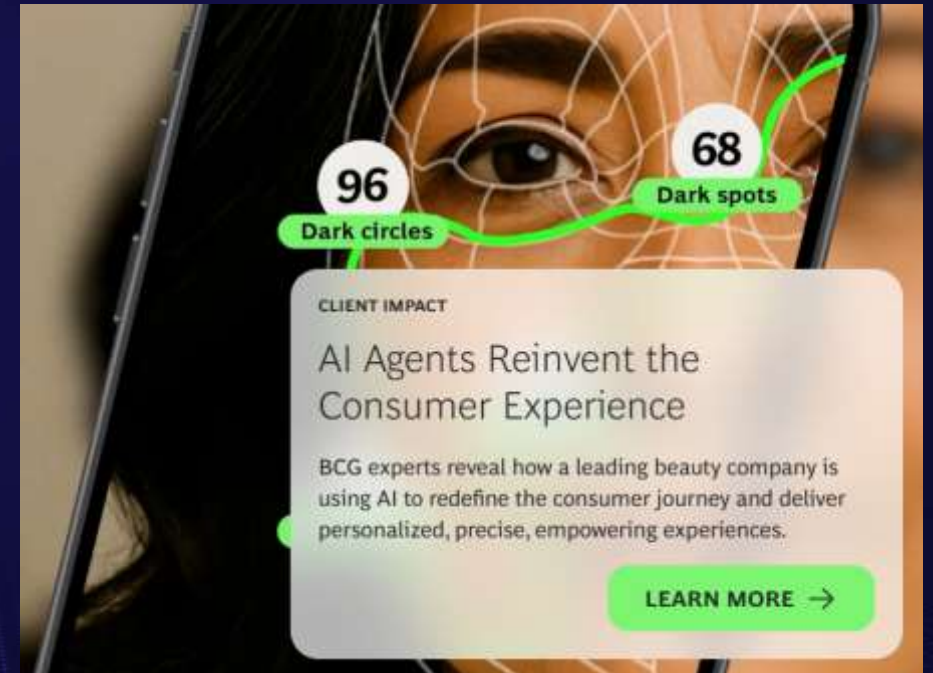


Guardrails : Protects the system

Caching : Makes the system faster, reduce latency, reuse tokens



Source : [BCG News Letter](#)





Guardrails are the safety mechanism that controls what goes into and comes out of AI Agents. They sit around your agent pipeline and ensure the agent :

1. Only processes safe, appropriate inputs
2. Only performs approved actions
3. Only returns validated and compliant outputs.

Guardrails help mitigate risks and protect you and your users. They should be placed where there are exposure to risks.

Types of Guardrails



Input Guardrail — Blocked

User asks: "How do I hack into a bank?" → Rejected before reaching the model.



Input Guardrail — Allowed

User asks: "Explain how online banking security works." → Safe, passes through.



Output Guardrail — Modified

Model response contains a real phone number → Stripped before sending to user.



Topic Guardrail — Out of Scope

Customer support bot asked about politics → Redirects back to product topics.

INTUITION



AI Agent

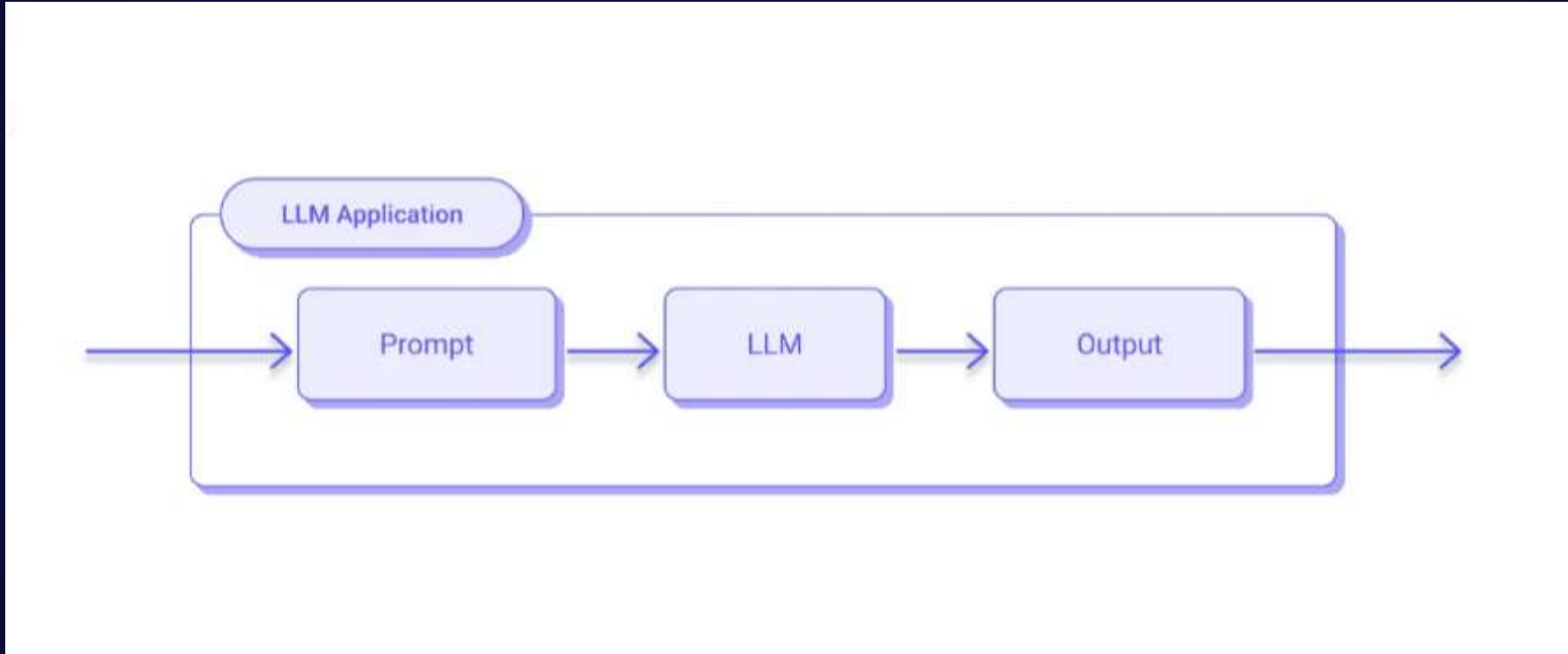


Think of guardrails like rules and fences on a road. They stop the agent from going into dangerous, harmful, or off-limits areas.

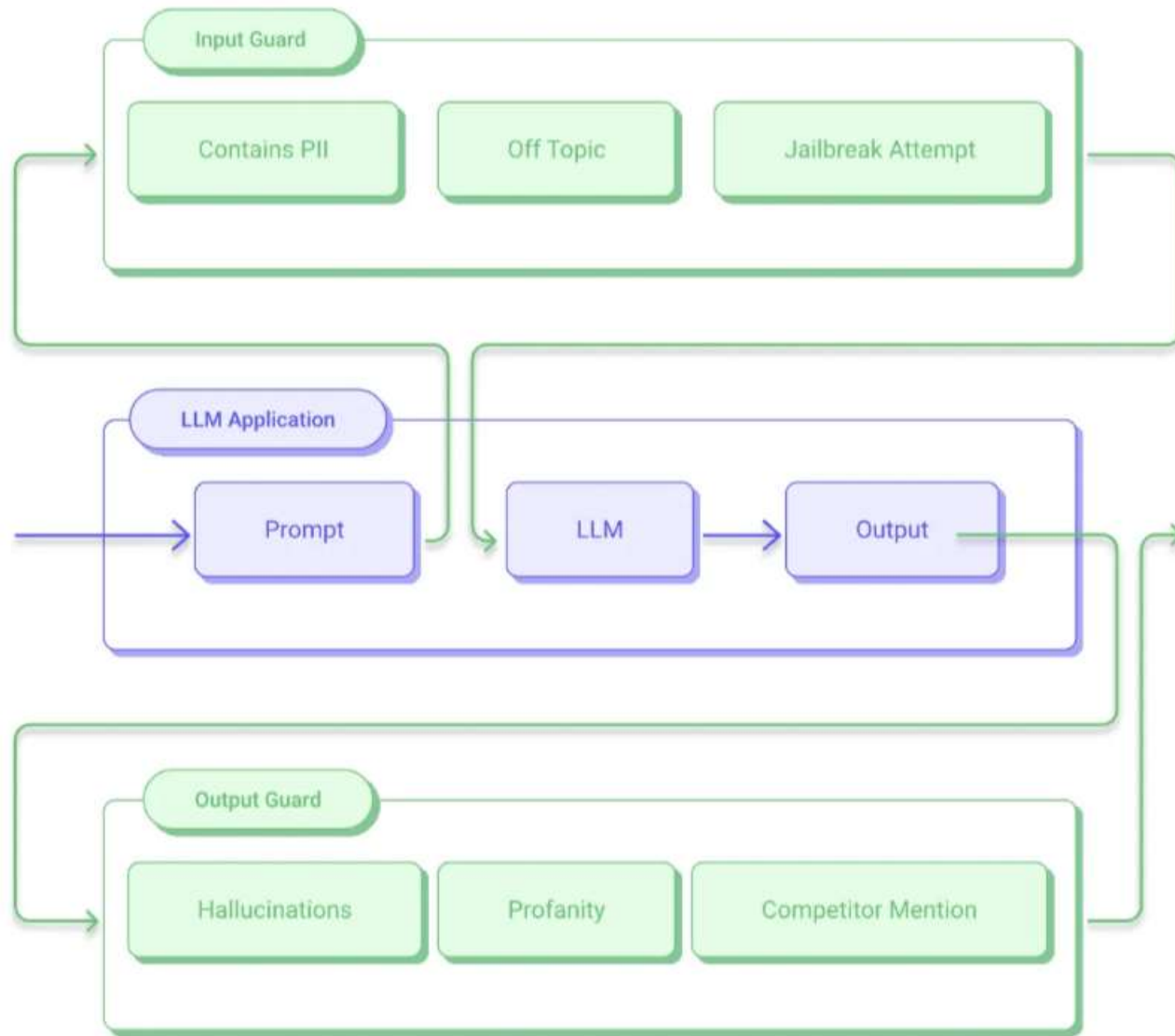


EXAMPLE (BANKING)

- If a user asks: "Give me someone's account details" Guardrail blocks it.
- If the agent tries to give financial advice, guardrail can restrict it to policy-approved info only.



With Guardrails





Guardrails



Deterministic guardrails

Use rule-based logic like regex patterns, keyword matching, or explicit checks. Fast, predictable, and cost-effective, but may miss nuanced violations.

✓ Strengths

- Fast
- Predictable
- Cost-effective
- Easy to implement

✗ Limitations

- May miss nuanced violations
- Less flexible
- Harder to handle complex context



Model-based guardrails

Use LLMs or classifiers to evaluate content with semantic understanding. Catch subtle issues that rules miss, but are slower and more expensive.

✓ Strengths

- Understands context
- Catches subtle issues
- More flexible
- Better for complex cases

✗ Limitations

- Slower
- More expensive
- Less predictable

GUARDRAILS WITH LANGCHAIN



Guardrails are implemented as Middleware in LangChain.



Middleware lets us add hooks at different points in the agent workflow.

Six Types of Middleware (Guardrails)

1 PII Middleware



Detects and handles Personally Identifiable Information (PII) in conversations.

- Detects emails, credit cards, IP addresses, and more
- Helps protect user privacy

2 Human in the Loop



Requires human approval before executing sensitive operations.

- Ideal for high-stakes decisions
- Useful in financial transactions, data changes, external communications, etc.
- Ensures accountability
- Includes Threads & Checkpoints (pause, resume, rollback)

3 Before Agent Hook



Validates requests at the start of each invocation.

- Session-level checks like authentication, rate limiting
- Blocks inappropriate requests before any processing
- Zero cost for blocked requests

4 After Agent Hook



Validates final outputs before returning to the user.

- Model-based safety checks
- Quality validation & compliance scans
- Can replace or mutate unsafe content
- Can use SLM as well

5 Layered Guardrails



Combines multiple middleware stacks together.

- Provides defense in depth
- Flexible and modular
- Stronger overall protection

6 (Optional) Custom / Other Hooks



Add any custom middleware at any point in the workflow.

- Domain specific checks
- Integration with external tools
- Extensible based on needs



KEY TAKEAWAY

Middleware guardrails help build safer, smarter, and more reliable AI agents by validating inputs, controlling actions, and ensuring safe outputs at every step.



Real Time Guardrails

<https://guardrails.openai.com/>

<https://github.com/openai/openai-guardrails-python>

<https://github.com/meta-llama/PurpleLlama>

<https://github.com/microsoft/PyRIT>

<https://github.com/NVIDIA-NeMo/Guardrails>

Code Demo

Prompt Caching

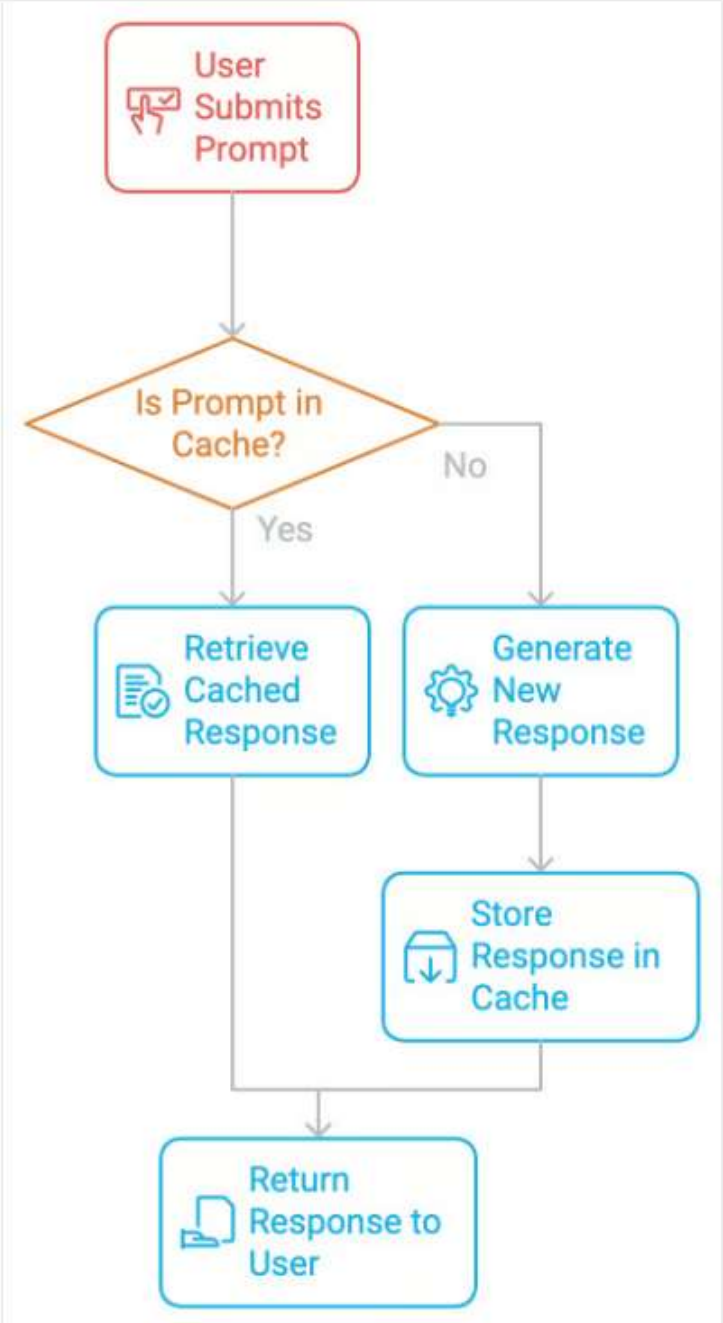
Prompt caching stores responses to frequently asked prompts.

This allows [language models](#) to skip redundant processing and retrieve pre-generated responses (cached tokens). It not only saves costs and reduces latency but also makes AI-powered interactions faster and more efficient.

Similar to other applications, the use of *caching* mechanisms can significantly help optimize LLM request costs and latency.

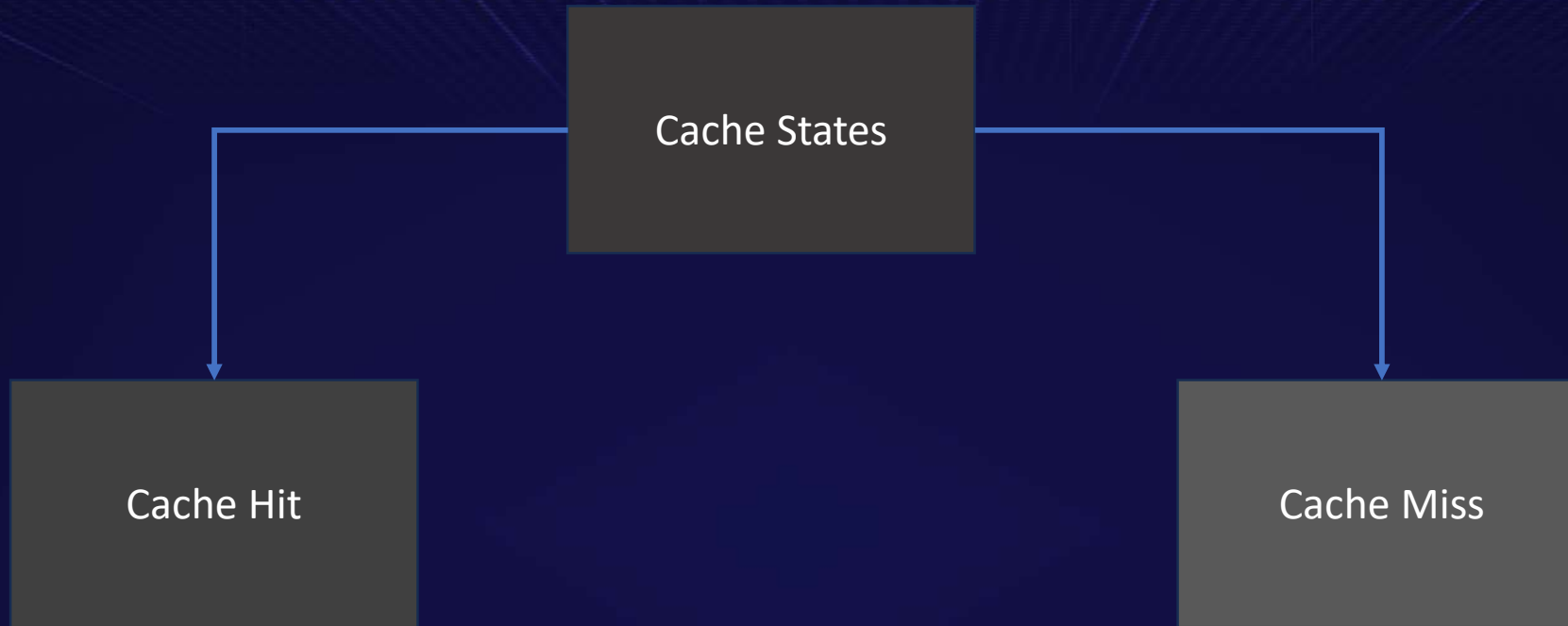
For instance, according to [OpenAI documentation](#), Prompt Caching can reduce latency by up to an impressive 80% and input token costs by up to 90%.

At its core, prompt caching works by storing prompts and their corresponding responses in a cache. When the same or a similar prompt is submitted again, the system retrieves the cached response rather than generating a new one. This avoids repetitive computation, speeds up response times, and reduces costs.



Why caching is important in Gen AI and Agentic AI ?

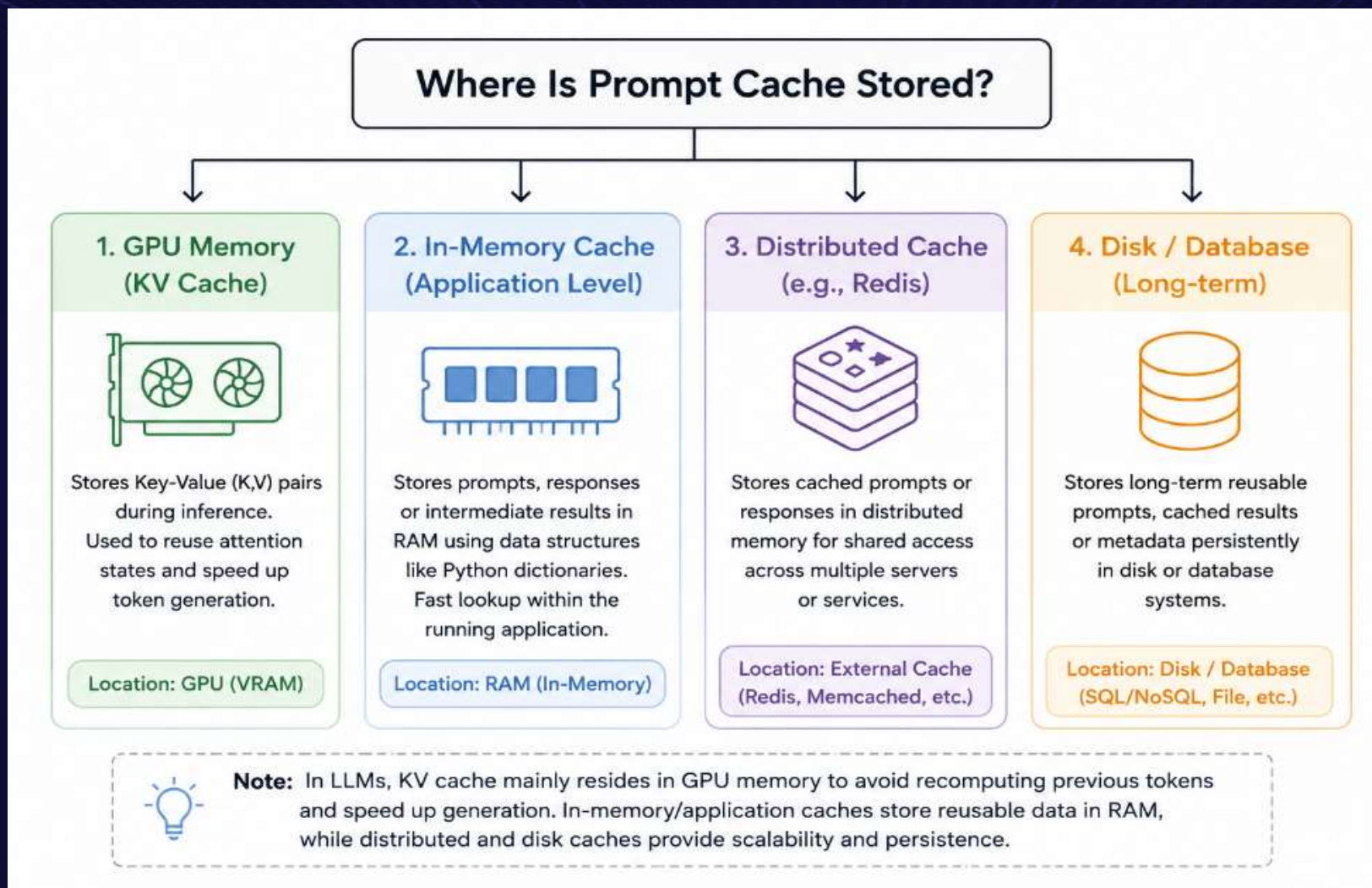
Prompt caching stores responses to frequently asked prompts.



A **cache hit** occurs when the requested data is found in the cache, allowing for a quick and cheap retrieval.

A **cache miss** occurs when the data is not in the cache, forcing the application to access the original source, which is more expensive and time-consuming.

Where Is Prompt Cache Stored?



Scenario: A Lawyer Using an AI Assistant

Setup:

A lawyer uploads a **50-page legal contract** and asks multiple questions about it.

The Conversation:

Lawyer: [50-page contract] → "Does this contract have a penalty clause?"

Lawyer: "What is the notice period mentioned?"

Lawyer: "Who are the two parties involved?"

Without KV Cache:

Every question → Model **re-reads all 50 pages** from scratch ❌

- Slow response
- High cost
- Wasteful computation

With KV Cache:

Question 1 → Model reads 50 pages, SAVES K&V in GPU VRAM

Question 2 → Loads saved K&V, only reads new question ✅

Question 3 → Loads saved K&V, only reads new question ✅

The **50-page contract** never gets re-processed again!

What is Cached Here?

What	Where
K & V of the 50-page contract	GPU VRAM
New questions (Q)	Always freshly computed

The contract is read once, its meaning is saved in GPU memory, and every new question just looks up that saved meaning like a bookmark in the model's brain.

You are a customer support bot for a pizza shop.

System Prompt (same every time):

"You are a pizza bot. We sell Margherita, Pepperoni, BBQ Chicken. Timings: 9am-11pm..."

Every user asks a different question:

- User 1 → *"What pizzas do you have?"*
- User 2 → *"What time do you close?"*

Without KV Cache → The model re-reads the entire system prompt for every user ✗

With KV Cache → The K & V of the system prompt are computed **once** and reused for every user. Only the new question is processed ✓

Code Demo

Conclusion

Guardrails keep bad data out of the cache and the model. Cache keeps cost and latency under control. Together they are what makes production-grade agentic AI viable.

Any Questions?

Thank You